DaeMon: A Frictionless, Containerized Distributed Compute Grid for Volunteer Resource Aggregation

ABSTRACT

Modern computational workloads—such as high-resolution artificial intelligence processing, cryptographic analysis, large-scale data transformations, and complex mathematical simulations—demand immense hardware resources, yet access to high-performance compute infrastructure remains severely constrained by the prohibitive financial expense, rigid vendor lock-in, and centralized architectural bottlenecks of commercial hyperscaler cloud platforms, while simultaneously billions of gigacycles of compute capacity sit completely dormant across personal workstations, laptops, and local enterprise edge devices. To resolve this systemic inefficiency and bridge the computational divide, DaeMon is engineered as a frictionless, containerized, volunteer-powered distributed compute grid that aggregates heterogeneous, idle hardware across networked environments to collaboratively execute massive polymorphic workloads at near-zero incremental infrastructure cost. The system operates through an active-push orchestration pipeline where administrators submit arbitrary computational tasks via an intuitive web dashboard to a centralized master orchestrator, which dynamically registers the job, executes domain-specific task-splitting algorithms to partition monolithic workloads into independent, atomic micro-tasks, and offloads heavy binary payloads directly to a dedicated distributed object storage cluster to ensure the message bus remains lightweight and responsive. Heterogeneous volunteer worker nodes, running as headless containerized background daemons across local or wide-area networks, establish connectivity, transmit high-frequency heartbeats, and receive targeted micro-task assignments pushed directly into dedicated asynchronous queues; upon assignment, each worker autonomously pulls its corresponding data slice from object storage, executes the specialized compute strategy within an isolated execution sandbox, uploads intermediate artifacts back to the storage cluster, and acknowledges completion. A fault-tolerant state aggregator continuously monitors node health, automatically redistributing stalled or orphaned tasks in the event of worker disconnection, intelligently reassembles and stitches finished micro-task artifacts into the final unified output, and broadcasts real-time telemetry, worker statistics, and job progress back to the user interface via persistent WebSocket channels. The end-to-end technology stack is constructed using Python with FastAPI for the high-performance asynchronous orchestrator and REST/WebSocket API engine, RabbitMQ as the enterprise message broker facilitating active task routing and worker queuing, Redis for real-time heartbeat caching and live worker registry management, MongoDB for flexible document persistence of polymorphic job metadata, MinIO for high-throughput self-hosted S3-compatible object storage, Docker for reproducible zero-configuration worker daemon containerization, and Next.js (React) paired with Tailwind CSS for the real-time monitoring and administrative command dashboard.

Keywords: Distributed Computing; Volunteer Computing; Task Partitioning; Fault-Tolerant Orchestration; Active Push Scheduling; Micro-Tasks; Message Queuing; RabbitMQ; FastAPI; Docker Containerization; MinIO Object Storage; Redis Heartbeat Registry; MongoDB; WebSockets; Next.js; Polymorphic Workloads.